

MANUAL DEL INTEGRANTE 2 — BACKEND 1 (ALGORITMOS Y OPTIMIZACIÓN)

Proyecto

Sistema de diseño y optimización de redes de fibra óptica utilizando algoritmos voraces, JavaFX y PostGIS.

1. OBJETIVO DEL MÓDULO

Este módulo será responsable de:

- Construir grafos.
 - Gestionar nodos y conexiones.
 - Ejecutar algoritmos voraces.
 - Optimizar la red.
 - Calcular costos.
 - Validar restricciones de distancia.
 - Controlar capacidad de postes.
 - Detectar saturación.
 - Sugerir expansión automática.
 - Comparar algoritmos.
-

NO ES RESPONSABILIDAD DE ESTE MÓDULO

No deberá trabajar con:

- PostgreSQL
 - PostGIS
 - SQL
 - JDBC
 - JavaFX
 - FXML
 - Leaflet
 - SHP
 - Interfaces gráficas
-

2. ESTRUCTURA DEL PROYECTO

```
backend1/  
├── graph/  
│   ├── Nodo.java  
│   ├── Arista.java  
│   ├── Grafo.java  
│   └── ResultadoRuta.java  
├── algorithms/  
│   ├── PrimAlgorithm.java  
│   ├── KruskalAlgorithm.java  
│   └── DijkstraAlgorithm.java  
├── services/  
│   ├── CostCalculator.java  
│   ├── DistanceValidator.java  
│   ├── ExpansionService.java  
│   └── NetworkOptimizationService.java  
└── enums/  
    ├── TipoNodo.java  
    └── EstadoNodo.java
```

3. FLUJO GENERAL

```
Frontend  
↓  
Backend 2 obtiene datos  
↓  
Backend 1 construye Grafo  
↓  
Usuario selecciona algoritmo  
↓  
PRIM o KRUSKAL  
↓  
Generación de red  
↓  
Validaciones  
↓  
Expansión automática  
↓  
ResultadoRuta
```

↓
Frontend

4. ENUM TipoNode

Ubicación:

enums/TipoNode.java

Valores:

CENTRAL,
POSTE_PRINCIPAL,
POSTE_SECUNDARIO,
CLIENTE,
SUGERIDO

5. ENUM EstadoNode

Ubicación:

enums/EstadoNode.java

Valores:

DISPONIBLE,
SATURADO,
INACTIVO

6. CLASE Nodo

Ubicación:

graph/Nodo.java

Responsabilidad:

Representar cualquier elemento de la red.

ATRIBUTOS

```
private int id;

private String nombre;

private TipoNodo tipo;

private EstadoNodo estado;

private int capacidadMax;

private int clientesActuales;

private double latitud;

private double longitud;
```

REGLA DE NEGOCIO

Todo poste debe controlar automáticamente su capacidad.

El Frontend NO debe calcular saturación.

MÉTODO

puedeConectar()

```
public boolean puedeConectar()
```

Retorna:

```
boolean
```

Implementación:

```
return estado != EstadoNodo.SATURADO;
```

MÉTODO

agregarCliente()

```
public boolean agregarCliente()
```

Retorna:

```
boolean
```

Implementación:

```
if(clientesActuales >= capacidadMax)
{
    return false;
}

clientesActuales++;

actualizarEstado();

return true;
```

MÉTODO

removerCliente()

```
public void removerCliente()
```

Retorna:

```
void
```

Implementación:

```
if(clientesActuales > 0)
{
    clientesActuales--;
}

actualizarEstado();
```

MÉTODO

capacidadDisponible()

```
public int capacidadDisponible()
```

Retorna:

```
int
```

Implementación:

```
return capacidadMax - clientesActuales;
```

MÉTODO PRIVADO

actualizarEstado()

```
private void actualizarEstado()
```

Retorna:

```
void
```

Implementación:

```
if(clientesActuales >= capacidadMax)
{
    estado = EstadoNodo.SATURADO;
}
else
{
    estado = EstadoNodo.DISPONIBLE;
}
```

MÉTODO

distanciaA()

```
public double distanciaA(  
    Nodo destino  
)
```

Retorna:

```
double
```

Función:

Calcular distancia entre nodos.

7. CLASE Arista

Ubicación:

```
graph/Arista.java
```

ATRIBUTOS

```
private Nodo origen;  
  
private Nodo destino;  
  
private double distancia;  
  
private double costo;
```

MÉTODO

calcularCosto()

```
public double calcularCosto()
```

Retorna:

double

Utiliza:

CostCalculator

8. CLASE Grafo

Ubicación:

graph/Grafo.java

ATRIBUTOS

```
private List<Nodo> nodos;  
  
private List<Arista> aristas;
```

MÉTODO

agregarNodo()

```
public void agregarNodo(  
    Nodo nodo  
)
```

Retorna:

void

MÉTODO

agregarArista()

```
public void agregarArista(  
    Arista arista  
)
```

Retorna:

```
void
```

MÉTODO

obtenerVecinos()

```
public List<Nodo> obtenerVecinos(  
    Nodo nodo  
)
```

Retorna:

```
List<Nodo>
```

9. CLASE ResultadoRuta

Ubicación:

```
graph/ResultadoRuta.java
```

ATRIBUTOS

```
private List<Arista> conexiones;  
  
private double costoTotal;  
  
private double distanciaTotal;
```

```
private int cantidadPostes;
```

MÉTODOS

```
public List<Arista> getConexiones()  
  
public double getCostoTotal()  
  
public double getDistanciaTotal()  
  
public int getCantidadPostes()
```

10. CONTROL DE CAPACIDAD

Objetivo:

Evitar la sobrecarga de postes.

REGLA

Si:

```
clientesActuales >= capacidadMax
```

Entonces:

```
estado = EstadoNodo.SATURADO
```

CONSECUENCIA

Los algoritmos NO deberán utilizar postes saturados.

Ejemplo:

```
if(poste.puedeConectar())  
{
```

```
    conectar();  
}
```

11. CLASE CostCalculator

Ubicación:

```
services/CostCalculator.java
```

RESPONSABILIDAD

Calcular el peso real utilizado por los algoritmos.

MÉTODO

calcularCosto()

```
public double calcularCosto(  
    Nodo origen,  
    Nodo destino,  
    double distancia  
)
```

Retorna:

```
double
```

FÓRMULA OFICIAL

```
double ocupacion =  
(double) origen.getClientesActuales()  
/  
origen.getCapacidadMax();  
  
double penalizacion =  
ocupacion * 100;  
  
return distancia + penalizacion;
```

COMENTARIO

La penalización evita utilizar postes casi saturados.

12. CLASE DistanceValidator

Ubicación:

```
services/DistanceValidator.java
```

RESPONSABILIDAD

Validar restricciones de distancia.

REGLAS

Conexión	Máximo
Central → Poste	150m
Poste → Poste	80m
Poste → Cliente	40m

MÉTODO

esValida()

```
public boolean esValida(  
    Nodo origen,  
    Nodo destino  
)
```

Retorna:

```
boolean
```

13. ALGORITMO PRIM

Ubicación:

```
algorithms/PrimAlgorithm.java
```

TIPO

Algoritmo Voraz.

OBJETIVO

Construir una red con costo mínimo.

MÉTODO

generarRed()

```
public ResultadoRuta generarRed(  
    Grafo grafo,  
    Nodo central  
)
```

Retorna:

```
ResultadoRuta
```

UTILIZA

```
CostCalculator  
DistanceValidator  
Nodo.puedeConectar()
```

14. ALGORITMO KRUSKAL

Ubicación:

```
algorithms/KruskalAlgorithm.java
```

TIPO

Algoritmo Voraz.

OBJETIVO

Generar una alternativa para comparar con Prim.

MÉTODO

generarRed()

```
public ResultadoRuta generarRed(  
    Grafo grafo  
)
```

Retorna:

```
ResultadoRuta
```

15. ALGORITMO DIJKSTRA

Ubicación:

```
algorithms/DijkstraAlgorithm.java
```

OBJETIVO

Calcular rutas específicas.

Ejemplo:

Central
↓
Cliente

MÉTODO

calcularRuta()

```
public ResultadoRuta calcularRuta(  
    Grafo grafo,  
    Nodo origen,  
    Nodo destino  
)
```

Retorna:

ResultadoRuta

16. CLASE ExpansionService

Ubicación:

services/ExpansionService.java

RESPONSABILIDAD

Sugerir postes cuando las distancias no sean válidas.

ESCENARIO

Poste
↓
120m
↓
Cliente

Máximo permitido:

40m

RESULTADO

Poste
↓
Sugerido
↓
Sugerido
↓
Cliente

MÉTODO

sugerirPostes()

```
public List<Nodo> sugerirPostes(  
    Nodo origen,  
    Nodo cliente  
)
```

Retorna:

List<Nodo>

IMPORTANTE

Los nodos generados deben tener:

TipoNodo.SUGERIDO

17. CLASE PRINCIPAL

Ubicación:

services/NetworkOptimizationService.java

RESPONSABILIDAD

Coordinar todos los algoritmos.

IMPORTANTE

Esta es la única clase que debe utilizar el Frontend.

MÉTODO

generarRed()

```
public ResultadoRuta generarRed(  
    Grafo grafo,  
    Nodo central,  
    String algoritmo  
)
```

Retorna:

ResultadoRuta

Implementación:

```
if(algoritmo.equals("PRIM"))  
{  
    return prim.generarRed(  
        grafo,  
        central  
    );  
}  
  
if(algoritmo.equals("KRUSKAL"))  
{  
    return kruskal.generarRed(  
        grafo  
    );  
}
```

```
    );  
}
```

MÉTODO

compararAlgoritmos()

```
public Map<String, ResultadoRuta>  
compararAlgoritmos(  
    Grafo grafo,  
    Nodo central  
)
```

Retorna:

```
Map<String, ResultadoRuta>
```

18. COMUNICACIÓN CON BACKEND 2

Backend 2 enviará:

```
List<Nodo>
```

Backend 1:

```
Construye Grafo  
Ejecuta algoritmos  
Genera ResultadoRuta
```

19. COMUNICACIÓN CON FRONTEND

GENERAR RED

```
networkService.generarRed(  
    grafo,  
    central,
```

```
);
```

COMPARAR ALGORITMOS

```
networkService.compararAlgoritmos(  
    grafo,  
    central  
);
```

20. IMPACTO VISUAL

Backend enviará:

```
EstadoNodo
```

Ejemplos:

```
DISPONIBLE  
SATURADO
```

El Frontend únicamente convertirá estados en colores.

Ejemplo:

```
DISPONIBLE → Verde  
SATURADO → Naranja  
SUGERIDO → Amarillo  
CENTRAL → Azul
```

21. GITHUB

Rama asignada:

```
backend-algorithms
```

Reglas:

- No modificar Frontend.
 - No modificar Backend 2.
 - No modificar Base de Datos.
 - Documentar métodos complejos.
 - Utilizar JavaDoc.
-

22. OBJETIVO FINAL

Al finalizar el módulo deberá ser posible:

- Construir grafos.
- Ejecutar Prim.
- Ejecutar Kruskal.
- Ejecutar Dijkstra.
- Calcular costos.
- Controlar capacidad.
- Detectar saturación.
- Validar distancias.
- Generar postes sugeridos.
- Comparar algoritmos.
- Integrarse con Backend 2.
- Integrarse con Frontend.